

DVT UNIT -4 Notes

Multivariate Data: Point-Based Techniques - Line- Based Techniques -Region-Based Techniques - Combinations of Techniques – Trees Displaying Hierarchical Structures –Graphics and Networks-Displaying Arbitrary Graphs/Networks.

Interaction concepts and techniques: Text and Document Visualization: Introduction- Levels of text representation- The Vector Space Model-Single document Visualizations- Document collection Visualizations- extended text visualizations

1. Point-Based Techniques:

Point plots are introduced as visualizations that project records from an n-dimensional data space to an arbitrary k-dimensional display space, such that data records map to k-dimensional points. For each record, a graphical representation, mark, or other aesthetic entity is drawn at its associated k-dimensional point. Individual visualization techniques identified as point plots define appropriate data projections and specific visual representations. Point plots can be defined to display individual records or summary records, and can be structured by various projection techniques.

(a) Scatterplots and Scatterplot Matrices

Scatterplots are one of the earliest and most widely used visualization techniques in data analysis. Both 2D and 3D scatterplots can be found in most packages designed to support data and information analysis. Their success stems from our innate abilities to judge relative position within a bounded space. As the dimensionality of the data increases, the choices for visual analysis consist of:

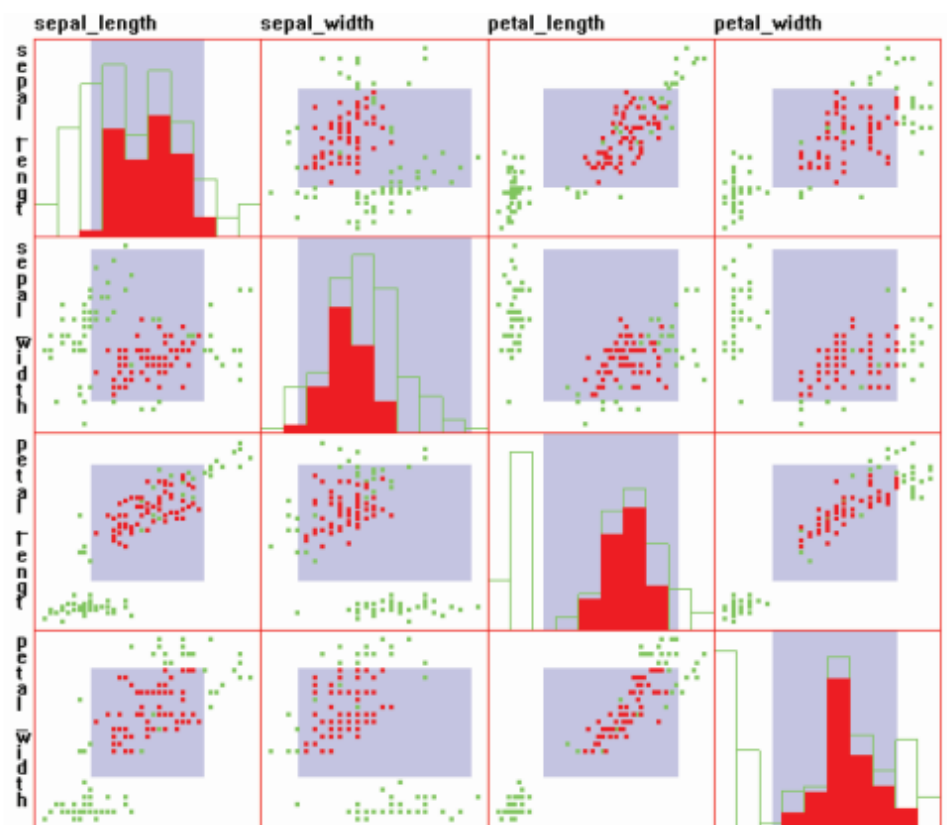
dimension subsetting—allowing the user to select a subset of the dimensions to display, or to develop algorithms to find the dimensions containing the most useful information for the task at hand.

- dimension reduction—using techniques such as principal component analysis or multidimensional scaling to transform the high-dimensional data to data of lower dimension, while attempting to preserve as best as possible the relationships among the data points

- dimension embedding—mapping dimensions to other graphical attributes besides position, such as color, size, and shape (though there are limits to how many dimensions can be included this way).

- multiple displays—showing, either superimposed or juxtaposed, several plots, each of which contains some of the dimensions.

For the case of multiple displays, the most common approach is to use a scatterplot matrix. This consists of a grid of scatterplots, with the grid having N^2 cells, where N is the number of dimensions. Thus, every pairwise plot will be shown twice, differing by a 90 degree rotation. The ordering of the dimensions is usually the same for the horizontal and vertical orientations, resulting in symmetry along the diagonal. The diagonal plots, which would normally plot a variable against itself, are often used to convey the names of the dimensions in the corresponding rows/columns, or sometimes to show a histogram of a given dimension.



A scatterplot matrix with the diagonal plot showing a histogram of each dimension. Note that the points and histogram regions in red indicate selected data.

(b) Force-Based Methods:

Many techniques for projecting high-dimensional points into 2D or 3D display space have been developed. The key goal is to attempt to maintain the N-dimensional features and characteristics of the data through the projection process, e.g., relationships that exist in the original data must also exist after projection. This, however, is not always possible, especially as the dimensionality of the data increases. The projection may also unintentionally introduce artifacts that may appear in the visualization and are not present in the data. In this section, we describe several such projection methods.

Multidimensional scaling is an important class of dimension reduction algorithms commonly used in statistical analysis and information visualization. The basic structure of a typical MDS algorithm is as follows:

1. Given a data set with M records and N dimensions, create an M by M matrix, D_s , that contains similarity measures between each pair of data items. For example, one might use Euclidean distance as a measure of similarity.
2. Assuming that you are projecting the data into K dimensions (e.g., for display purposes, K is usually between 1 and 3), create an M by K matrix, L , to contain the locations for the projected points. These M locations can initially be randomly chosen, or techniques such as principal component analysis (PCA) can be used to create reasonable initial positions.
3. Compute an M by M matrix, L_s , that contains the similarities between all pairs of points in L .
4. Compute the value of stress, S , which is a measure of the differences between D_s and L_s . Many such stress measures exist; most assume that the coordinate systems have been normalized so that the maximum distance between points is 1.0.
5. If S is sufficiently small, or hasn't changed significantly in recent iterations, the algorithm terminates.
6. Otherwise, attempt to shift the positions of points in L in a direction that will reduce their individual stress levels. For example, this might be a weighted sum of displacements based on comparing the point with all other points, or perhaps only with its nearest neighbors. The displacement should be scaled such that points don't oscillate between positions.
7. Return to step 3

As one can imagine, there are many possible variants on this algorithm, including different similarity and stress measures, different initial and termination conditions, and different position update strategies. As in any optimization process, there is the potential to fall into a local minimal configuration that still has a high level of stress. Common strategies to alleviate this include occasionally adding a random jump in the position of a point to see if it will converge to a different location.

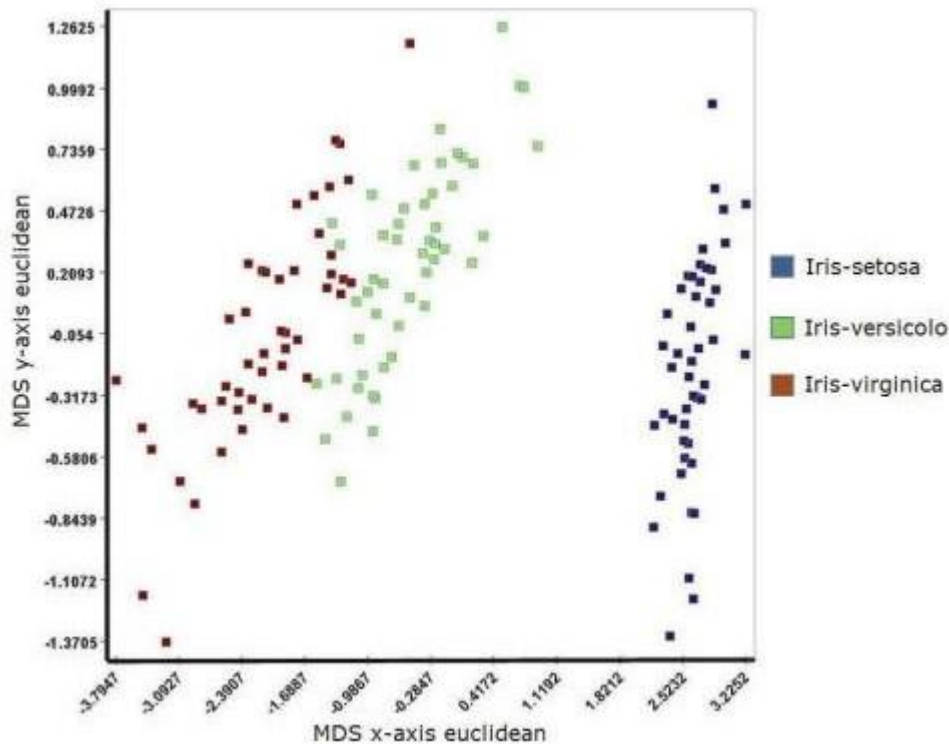


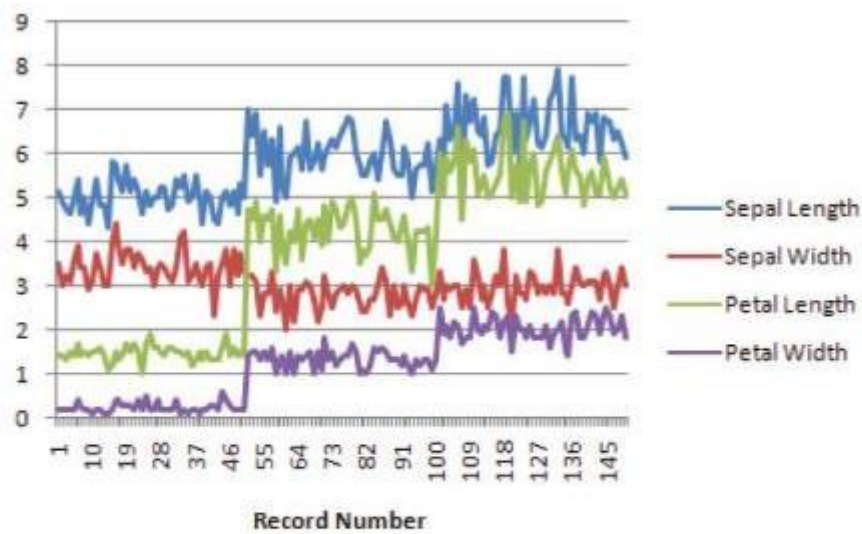
Fig 8.2 Iris data set projected using MDS

2. Line-Based Techniques:

Point-based methods represent each data value or record with a small mark. In line-based methods, points corresponding to a particular record or dimension are linked together with straight or curved lines. These lines not only reinforce the relationships among the data values, but also convey perceivable features of the data via slopes, curvature, crossings, and other line patterns. We describe a couple of techniques within this class of methods.

A) Line Graphs:

A line graph is a univariate visualization technique where the vertical axis represents the range of values for the variable and the horizontal axis represents some ordering of the records in the data set. Most univariate visualization techniques can be extended to multivariate data by either superimposing or juxtaposing the visual representations of individual variables. A commonly used method of this sort is via line graphs. For a modest number of data dimensions, the line plots can be drawn on a common set of axes, differentiating the dimensions using color, line style, width, or other graphical attributes (see Figure).



An example of a multivariate line chart, in this case, the four-dimensional Iris data set. Note that the modest separation between dimensions makes it easy to identify trends and outliers.

When the number of dimensions increases, or the dimensions have significant overlap in their data ranges, superimposing becomes more.

Figure 8.7(a) shows an 8-dimensional data set (AAUP faculty salaries and compensation for different ranks at 100 universities). Clearly, patterns are difficult to perceive. However, a few strategies are possible. Figure 8.7(b) shows a stacked line chart, where instead of using a common baseline, each dimension uses the plot from the previous dimension as a base. While this reduces the number of occlusions, it makes it more difficult to assess actual values of data points. Figures 8.7(c) and 8.7(d) show the effect of another strategy, namely, sorting the records based on one of the dimensions. Patterns in both the regular and stacked versions are easier to identify.

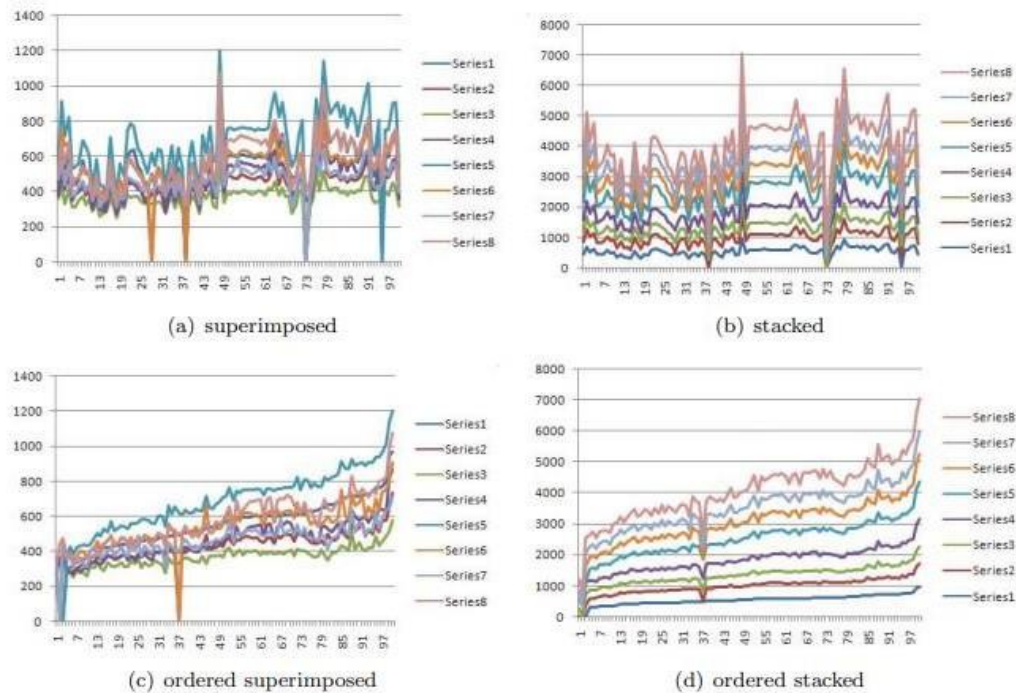


Figure 8.7. Four versions of line graphs for a subset of the AAUP data set: superimposed, stacked, ordered superimposed, and ordered stacked. Ordering is based on the first dimension, which represents salaries of full professor.

B) Parallel Coordinates

Parallel coordinates, also called $||$ -coords and PCP (for parallel coordinates plot), were first introduced by Inselberg in 1985 as a mechanism for studying high-dimensional geometry. Since then, numerous researchers have studied and enhanced PCPs for use in multivariate data analysis. The basic idea is that axes, rather than being orthogonal, are parallel, with evenly spaced vertical or horizontal lines representing a particular ordering of the dimensions. A data point is plotted as a polyline that crosses each axis at a position proportional to its value for that dimension. Figure 8.8 shows an example. One could consider a PCP as a line graph after rotating the data, since the values of a record are linked together, as opposed to the values of a dimension.

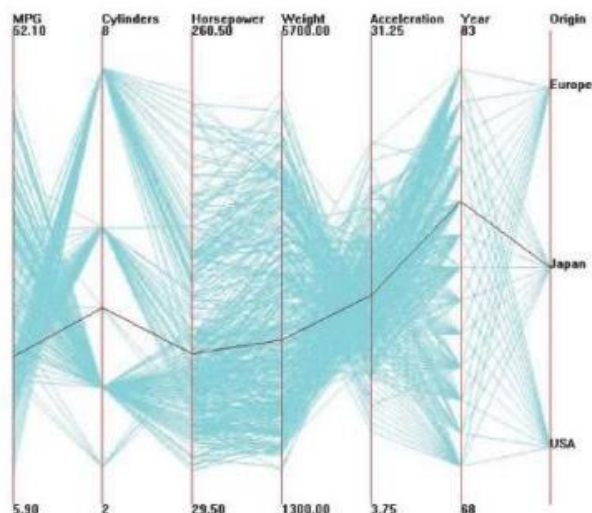


Figure 8.8. An example of a 7-dimensional data set visualized with parallel coordinates. A single data point is represented as the darkened polyline.

C) Andrews Curves

Another line-based visualization for multivariate data is the Andrews curve, developed by David F. Andrews in 1972. Each multivariate data point $D = (d_1, d_2, \dots, d_N)$ is used to create a curve of the form

$$f(t) = \frac{d_1}{\sqrt{2}} + d_2 \sin(t) + d_3 \cos(t) + d_4 \sin(2t) + d_5 \cos(2t) + \dots$$

For an odd number of dimensions, the final term is $d_N \cos(N-1/2 t)$, while for an even number of dimensions the final term is $d_N \sin(N/2 t)$. As in many multivariate visualization techniques, the order of the dimensions can have a significant effect on the resulting Andrews curve. Figures 8.10(a) and 8.10(b) show the same data with different dimension orders. In particular, outliers may become more or less perceivable, depending on the ordering

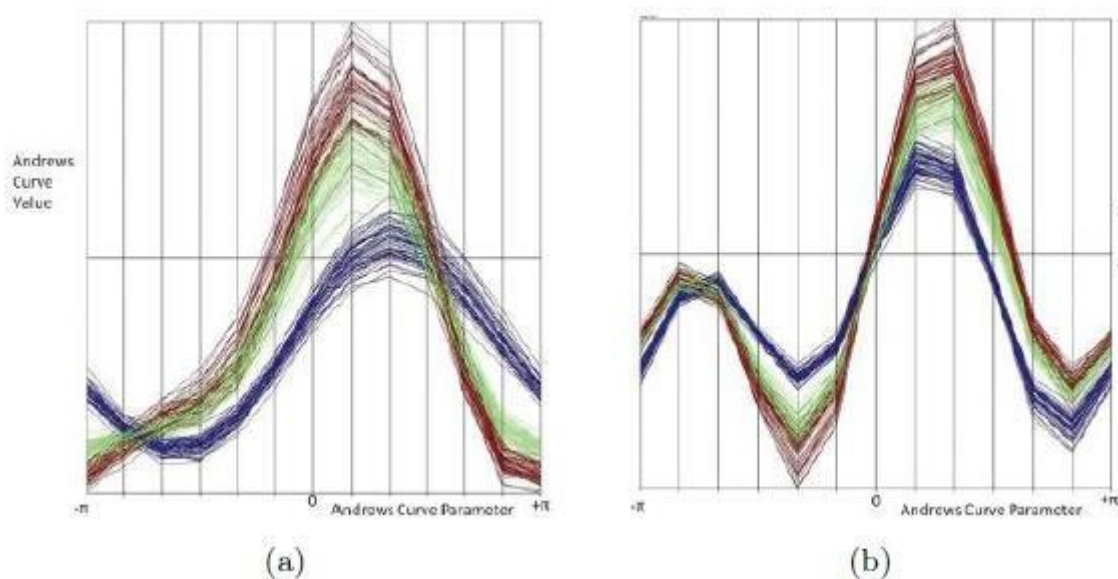


Fig 8.10

An example of Andrews curves using two different dimension orders: (a) based on the original order of the dimensions (sepal length, sepal width, petal length, petal width); (b) based on the original order of the dimensions in reverse order.

D).Radial Axis Techniques

For each of the techniques that orient the coordinate systems horizontally and/or vertically, there is an equivalent technique that uses a radial orientation. For example, a circular line graph is one in which the plotted lines are offset from a circular base (see Figure). A long graph can be nested by dividing it up into equal size segments and mapping each to a base of different radius. This is a potentially useful way to study cyclic events. Variants on circular line graphs include radar and star graphs.

Many other circular charts have been developed over the years, including:

1. polar graphs—point plots using polar coordinates;
2. circular bar charts—like circular line graphs, but plotting bars on the base line
3. circular area graphs—like a line graph, but with the area under line filled in with a color or texture;
4. circular bar graphs—with bars that are circular

arcs with a common center point and base line (note the difference between these and circular bar charts: in one, the bar is straight and the base is curved, but vice versa for the other).

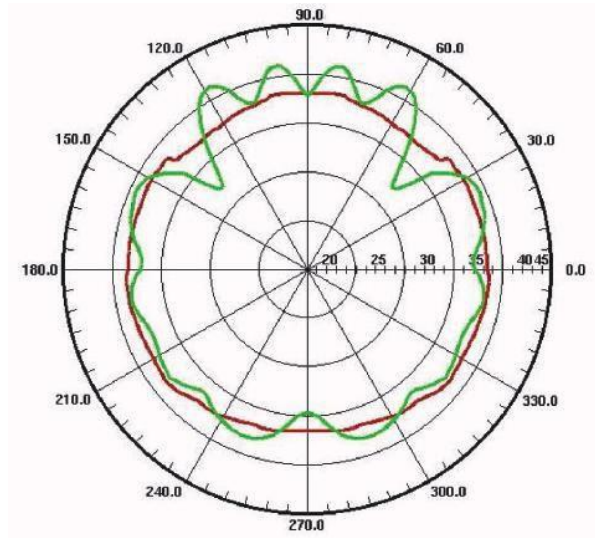


Fig: 8.11 An example of a circular line graph.

3. . Region-Based Techniques

In region-based techniques, filled polygons are used to convey values, based on their size, shape, color, or other attributes. Even though it was mentioned in Chapter 3 that our ability to accurately measure area is noticeably worse than our ability to measure attributes such as length, many effective techniques in this category have been developed. For some, the goal is not to show the raw data itself, but rather summaries or distributions of the values. Many of the region-based techniques were initially designed for univariate data, such as pie charts and bar charts. Some, however, have been extended to multiple dimensions

A. Bar Charts/Histograms

One of the most common visualizations, in addition to line plots, scatterplots, and maps, is the bar chart, where rectangular bars are used to convey numeric values. Humans have high visual acuity when it comes to comparing the length of linear features. Thus, bar charts are a natural choice for visualizing many kinds of data. Both horizontal and vertical bars are routinely used and are readily interpretable, which means that the visualization designer has some flexibility in the way they are integrated into a given application. If each bar is to receive a text label, it is easier to use horizontally oriented bars, but angling the text for vertical bars can also help to solve the problem of lengthy strings

One of the critical decisions that needs to be made when using bar charts is deciding how many bars are needed to best represent the data. If the bars represent the state of N variables, then as long as N is not too large, there can be a one-to-one correspondence between variables and bars. If the goal is to represent a summarization or distribution of a data set, we can use a histogram to convey the number of occurrences of data values. If the data takes on nominal values or a modest number of distinct integers, the decision

is simple— just have the same number of bars as there are different values. For continuous data or integer variables with a large range, we need to divide the data into subranges and assign each subrange to a bar.

If the data is multivariate, there are several options for using bar charts. A common alternative is a stacked bar graph, where each bar consists of several shorter bars to represent the values for each dimension (Figure 8.13a). It is common to vary the color, texture, or other attributes of the bars to make them readily distinguishable within a given bar and comparable between bars.

Similarly, bars for different variables can be placed next to each other, giving them a common baseline and thus simplifying their interpretation (Figure 8.13b). The choice between these approaches is often based on the number of variables, as well as the number of bars. Stacked bars distribute the burden in two directions, while adjacent bars can demand significant space, unless the bar and dimension counts are modest.

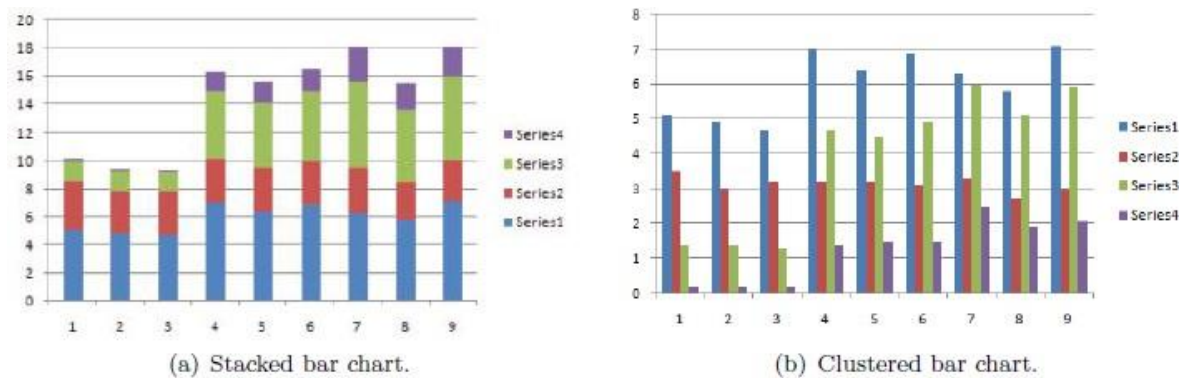


Figure 8.13. Examples of 2D bar graphs for showing multivariate data.

Cityscapes are a 3D version of bar graphs, where 3D boxes are used instead of 2D rectangles. Bars are laid out on a grid using two data dimensions to position the bar on the surface. Other variables control the size and color of the boxes at a given location. Cityscapes get their name.

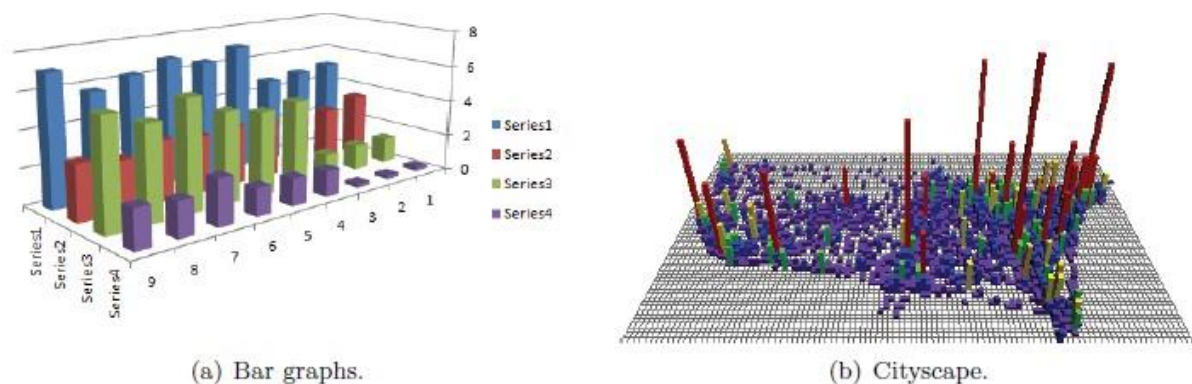


Figure 8.14. Examples of 3D visualizations for showing multivariate data.

Cityscapes are a 3D version of bar graphs, where 3D boxes are used instead of 2D rectangles. Bars are laid out on a grid using two data dimensions to position the bar on the surface. Other variables control the size and color of the boxes at a given location. Cityscapes get their name from the fact that the resulting visualization often looks like the buildings of a city. If all cells in the grid are populated, the graph is sometimes called a 3D histogram. The problem with going to 3D is that there is now the possibility of occlusion (Figure 8.14a). Several strategies exist for reducing or eliminating this occlusion. One is to provide the user the option to rotate the view, thus exposing bars that were previously blocked. Another is to shrink the thickness of the bars to reduce the size of their footprints and therefore reduce the number of bars that a given bar can block. A third approach is to vary the opacity of bars, allowing one to see the bars blocked by those closer to the viewpoint. All of these approaches have shortcomings, but even with their limitations, cityscapes are a popular visualization technique, especially with georeferenced data (Figure 8.14(b)).

B. *Tabular Displays*

Multivariate data is often stored in tables, and a number of visualization techniques have been modeled on this structure. These techniques mostly vary in the types of interactions they support. Heatmaps are created by displaying the table of record values using color rather than text. For this visualization technique, all data values are mapped to the same normalized color space, and each is rendered as a colored square or rectangle. Using different color maps, as well as allowing users to stretch or compress colors to emphasize or deemphasize some value.

C. *Dimensional Stacking*

Dimensional stacking is a technique used to show high-dimensional (multi-variable) data on a 2D image without losing much spatial information or overlapping data points.

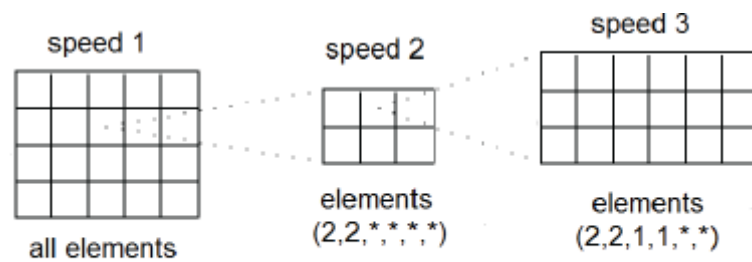
In simple terms:

- The data has many dimensions (variables).
- Two dimensions are shown first as a small 2D grid (like rows and columns).
- Inside each cell of that grid, another smaller grid is drawn for the next two dimensions.
- This process repeats until all dimensions are represented.

Each dimension changes or “moves” at a different rate, similar to how the digits on an odometer turn at different speeds.

In the end, every data point in the high-dimensional space is mapped to a unique spot on the 2D image.

The value of the dependent variable at the location in the high-dimensional space is then mapped to a color/intensity value at that location in the two-dimensional image. This embedding process is illustrated in Figure 8.18 with a six-dimensional data set, where dimensions $d_1, \dots, d_6$ have cardinalities 4, 5, 2, 3, 3, and 6, respectively. For clarity, we have not displayed the values associated with a dependent variable, which would be the seventh dimension and would dictate the colors in the smallest grid locations. Figure 8.19 is an example of a dimensional stacking visualization.



Conceptualization of dimensional stacking; collapsing six dimensions into two dimensions.

Figure 8.18.

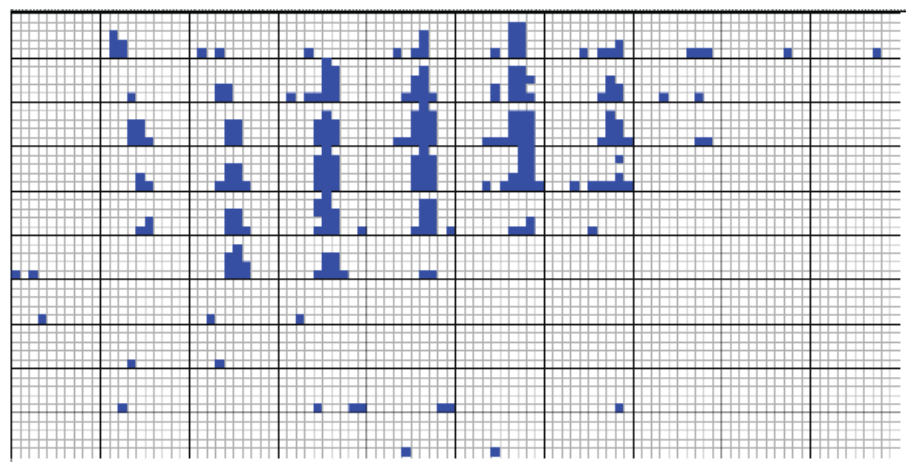


Figure 8.19.

An example of 4D data visualized using dimensional stacking. The data consists of drill-hole data, with three spatial dimensions, and the ore grade as the fourth dimension.

Combinations of Techniques

In addition to the techniques based on points, lines, or regions, there are a number of hybrid techniques that combine features of two or more of these classes. We describe two of the more popular techniques of this type: glyphs and dense pixel displays.

Glyphs and Icons

In the context of data and information visualization, a glyph is a visual representation of a piece of data or information where a graphical entity and its attributes are controlled by one or more data attributes. As an example, the width and height of a box could be controlled by a student's score on the midterm and final exam for a course, while the color could be associated with the gender of the student. This is a rather broad definition for the term, as it can cover such visual elements as the markers in a scatterplot, the bars of a histogram, or even an entire line plot.

Data values in visualization can be represented through various graphical attributes such as **position, size, shape, orientation, color or texture, line style, and motion**.

There are three main ways to map data to these attributes:

1. **One-to-one mapping** – each data value controls a different visual feature.
2. **One-to-many mapping** – the same data value is shown using multiple visual cues for better clarity.
3. **Many-to-one mapping** – multiple data values are represented through variations of a single visual attribute (like position or color).

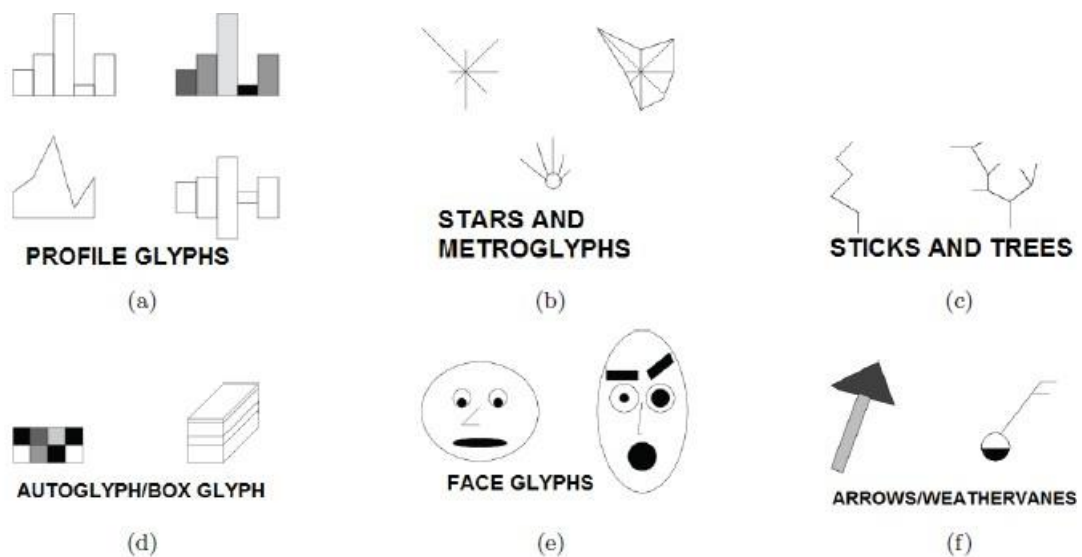


Figure 8.20. Examples of multivariate glyphs (from [445]).

One-to-one mappings are often designed to take advantage of the user's domain knowledge, using intuitive pairings of data to graphical attributes to ease the learning process. Examples include mapping color to temperature, and flow direction to line orientation. Redundant mappings can be useful in situations where the number of data dimensions is low and the desire is to reduce the possibility of misinterpretation. For example, one might map population to both size and color to ease analysis for color-impaired users, and to facilitate comparison of two populations with similar values. Many to-one mappings are best used in situations where it is important to not only compare values of the same dimension for separate records, but also to compare different dimensions for the same record. For example, mapping each dimension to the height of a vertical bar facilitates both intra-record and inter-record comparison.

In using glyphs for information visualization, we need to be aware of the many biases and limitations of the technique. First and foremost are the perceptual biases, depending on what graphical attributes are being used. There are some attributes, such as the length of a line, that we can judge more accurately than others, such as orientation or color. Other issues of bias include the fact that relationships between adjacent graphical attributes are much easier to perceive than those that are more distant, with a few notable exceptions. Similarly, comparing two glyphs that are near each other on the screen is easier than if the glyphs are more separated. Finally, the number of data dimensions and records that can be effectively handled with glyphs is limited.

Once a glyph design is chosen, there are $N!$ different dimension orderings that can be used in the mapping. Which ones are likely to reveal the most interesting features? Several ordering strategies can be imagined:

- Dimensions could be ordered according to their correlation, so that similar dimensions are

mapped adjacent to each other. This can help reveal general trends, as well as expose some outliers.

- Dimensions can be mapped in such a way as to promote symmetrically shaped glyphs, which can be easier to perceive and remember. Shapes that are less symmetric than their neighbors will also stand out.

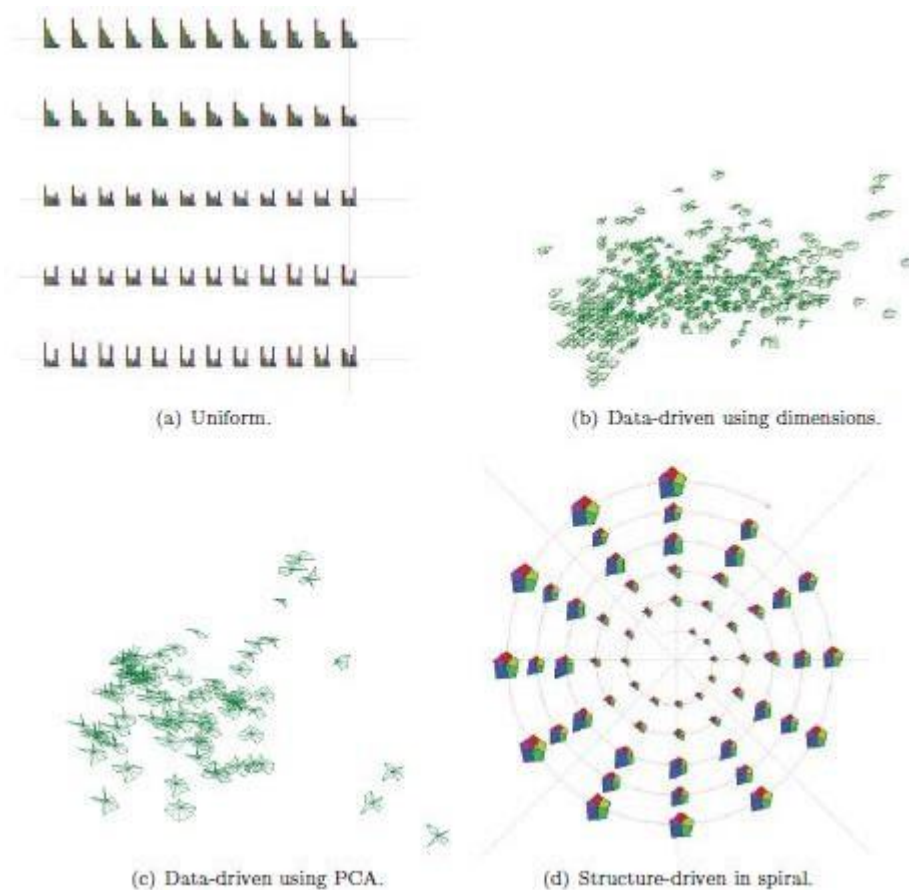
- Dimensions can be sorted according to their values in one record.

For example, if the data represents a multivariate time series, sorting the dimensions based on the first record can make the trends over time more pronounced, conveying which dimensional relationships were maintained versus those that changed significantly.

- Dimensions can be manually sorted, based on the user's knowledge of the domain. Thus, semantically similar dimensions can be grouped or used for symmetric glyph features, which can simplify the interpretation.

A final important consideration in designing a glyph-based visualization is the layout of the glyphs on the screen. There are three general classes of layout strategies:

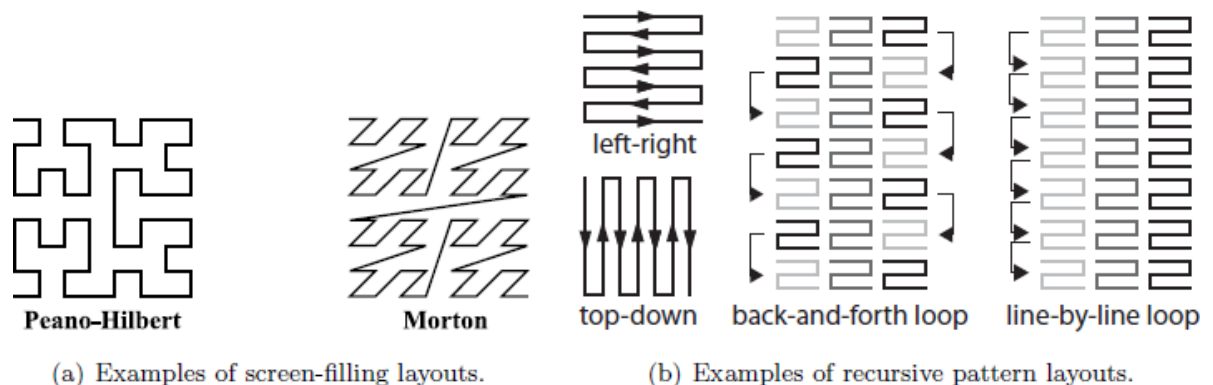
1. uniform—glyphs are scaled and positioned with equal space between them to occupy the entire screen. This strategy eliminates overlaps, while making efficient use of the screen space. Different orderings of records can expose different data features in the same way that different dimension orderings can (Figure 8.21(a)).
2. data-driven—data values are used to control the positioning of the glyphs. Two approaches are possible. In the first, two data dimensions (or three for 3D display) are chosen to set the locations (Figure 8.21(b)), while in the second, positions are derived from the data values using algorithms such as PCA (Figure 8.21(c)) or MDS. In either case, we can apply one or more additional passes over the data to reduce overlaps by separating glyphs that are too close.
3. structure-driven—if the data has an implicit or explicit structure to it, such as cyclic or hierarchical, this can be used to control the positioning. For example, glyphs may be laid out in a spiral or a grid to emphasize cyclic patterns (Figure 8.21(d));



Dense Pixel Displays

Dense pixel displays (also known as pixel-oriented techniques) are a hybrid between point-based and region-based methods. Pioneered by Keim and his colleagues, these techniques map each value to individual pixels and create a filled polygon to represent each data dimension. The displays make maximal use of the screen space, allowing data sets with millions of values to be shown on a single screen. Each data value will control the color for a single pixel; changing the color map used can potentially reveal new features of the data. Given a data set and a color map, the issues that remain to be resolved are the layout of the data records and their ordering.

Examples of pixel layout patterns in dense pixel displays



Displaying Hierarchical Structures

Trees or hierarchies (we'll use the terms interchangeably) are one of the most common structures to hold relational information. For this reason, many visualization techniques have been developed for display of such information. We can divide these techniques into two classes of algorithms: space-filling and non-space-filling. The rest of this section will provide details on implementing algorithms for visualizing this type of data.

A. Space-Filling Methods

As the name implies, space-filling techniques make maximal use of the display space. This is accomplished by using juxtapositioning to imply relations, as opposed to, for example, conveying relations with edges joining data objects. The two most common approaches to generating space-filling hierarchies are rectangular and radial layouts.

Treemaps and their many variants are an alternative representation of a Venn diagram and are the most popular form of the rectangular spacefilling layout. In the basic treemap, a rectangle is recursively divided into slices, alternating horizontal and vertical slicing, based on the populations of the subtrees at a given level. An example is shown in Figure 9.2.

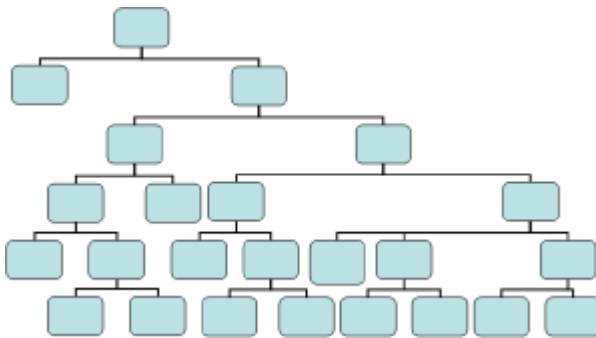


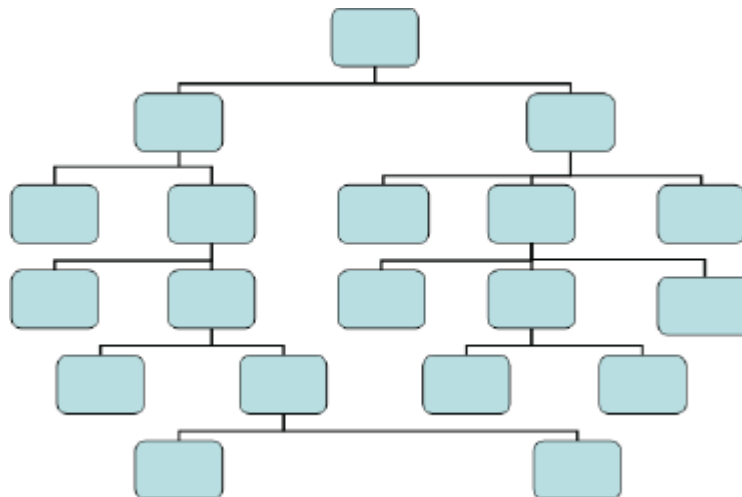
Figure 9.2

The methods described above are structured using horizontal and vertical divisions to convey the hierarchy. A number of other approaches are possible, however, such as those that divide space radially. Radial space-filling hierarchy visualizations, sometimes referred to as sunburst displays, have the root of the hierarchy in the center of the display and use nested rings to convey the layers of the hierarchy. Each ring is divided based on the number of nodes at that level. These techniques follow a similar strategy to treemaps, in that the number of terminal nodes in a subtree determines the amount of screen space that will be allocated for it.

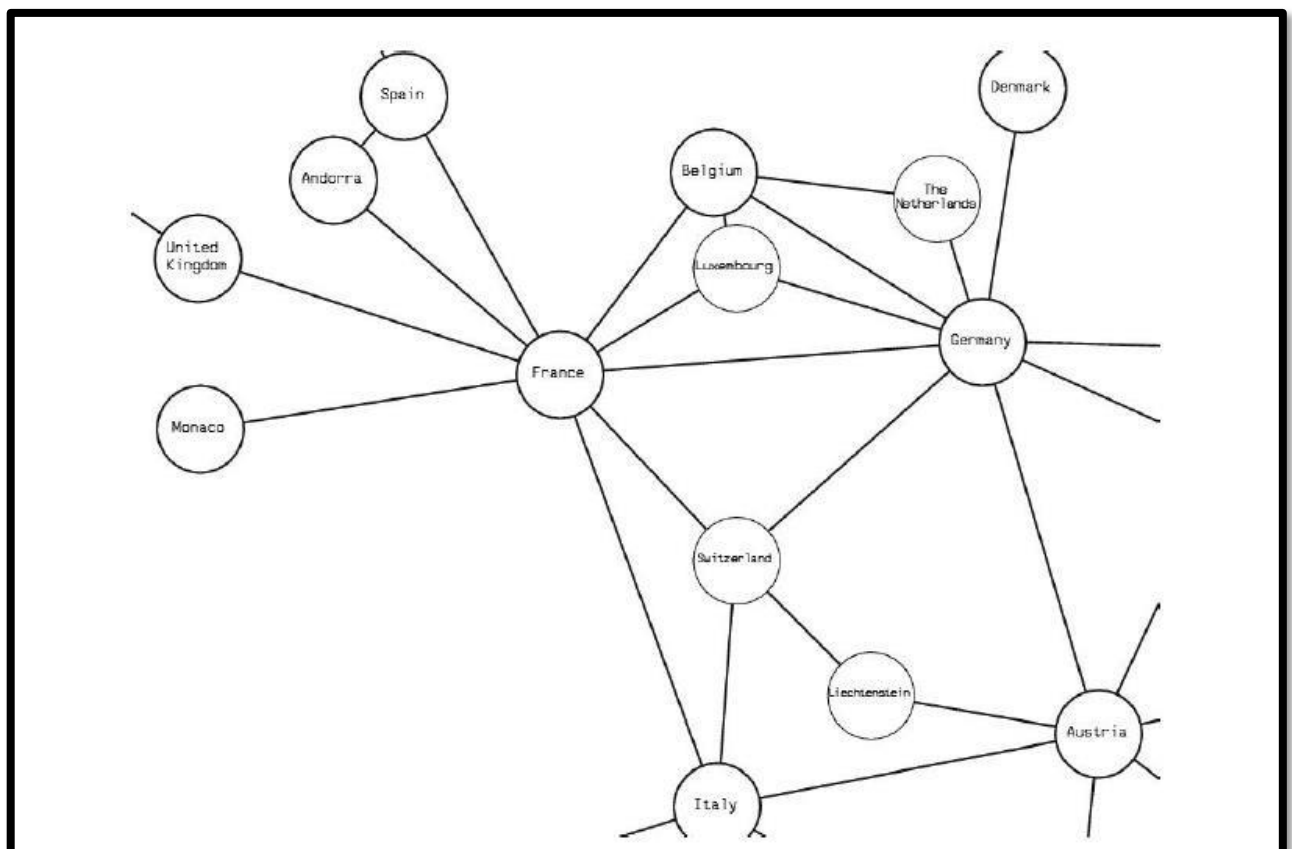
B. Non-Space-Filling Methods

The most common representation used to visualize tree or hierarchical relationships is a node-link diagram. Organizational charts, family trees, and tournament pairings are just some of the common applications for such diagrams. The drawing of such trees is influenced the most by two factors: the fan-out degree (e.g., the number of siblings a parent node can have) and the depth (e.g., the furthest node from the root). Trees that are significantly constrained in one or both of these aspects, such as a binary tree or a tree with only three or four levels, tend to be much easier to draw than those with fewer constraints.

An example of visualizing hierarchies with a simple node-link diagram, using equal spacing per level.



Displaying Arbitrary Graphs/Networks



Trees are just one type of a more general representation of relations called graph. Technically speaking, a tree is a connected, unweighted, acyclic graph. Clearly, there are many other possibilities, including graphs with weighted edges, undirected graphs, graphs with cycles, disconnected graphs, and so on. Rather than give more algorithms specific to other classes of graphs, which could certainly fill more than a textbook, we will describe some general approaches for visualizing graphs in which the class or structure is not known, which we term an arbitrary graph. For our purposes, we will assume that the graph is undirected, though some of the techniques

presented are easily extended to directed graphs. We will look at two distinct graph drawing approaches: node-link diagrams (building on the material from the previous section) and matrix displays.

Node-Link Graphs:

Force-directed graph drawing methods use a spring analogy to represent the links, with node positions iteratively refined until the overall energy or stress of the system is minimized. For each pair of connected nodes, there are two forces: f_{ij} , the force caused by the spring between them, and g_{ij} , an electrical repulsion force to keep nodes from getting too close.

A simple model is to use Hooke's law to represent the spring force and an inverse square law to represent the repulsion force. If $d(i, j)$ is the Euclidean distance between nodes i and j , $s_{i,j}$ is the natural spring length (at rest), and k_{ij} is the spring tension, the x-component of the spring force between two nodes can be computed as

$$f_{ij}(x) = k_{ij} * (d(i, j) - s_{ij}) * (x_i - x_j) / d(i, j).$$

If r_{ij} is the strength of the repulsion between nodes i and j , the x-component of the repulsion force can be computed as

$$g_{ij}(x) = (r_{ij} / d(i, j)^2) * (x_i - x_j) / d(i, j).$$

Thus, one step of the position refinement process would calculate the sum of all the forces on each node (x-, y-, and z-components, as appropriate) and move its position proportional to that force. Clearly, once points have moved, all the forces need to be recalculated and another shift of positions made. To avoid oscillation, it is common to start with movements that are a significant percentage of the force and then use smaller and smaller step sizes to converge on the point where the forces are minimized. Initial positions can be assigned randomly. As it is quite possible to end up in a local, rather than a global, energy minimum, it is common to run the layout algorithm Visualization Techniques for Trees, Graphs, and Networks

An example of a force-directed graph. The graph, showing relationships between countries of Europe, was generated with aiSee: <http://www.aisee.com>.

multiple times with different initial configurations to find the best of several computed configurations. The goodness of the layout can be computed based on the sum of the magnitude of forces on a given configuration.

Planar graph drawing techniques start with the assumption that the underlying graph is planar, e.g., it has no edge crossings. These algorithms have gotten a lot of attention, for several reasons. First, as the theory of planar graphs has a long history, there are many concepts that can be exploited from the literature. Second, as edge-crossings tend to make graphs difficult to read, it is a good strategy to minimize or eliminate such crossings. Finally, planar graphs tend to be sparse; Euler's formula indicates that a planar graph with n vertices has at most $3n - 6$ edges. Concentrating on planar graphs is not overly restrictive, as one can eliminate crossings by inserting dummy nodes at the crossings, perform the layout using a planar graph algorithm, and then remove the dummy nodes.

We will, in addition, assume that the graph is connected, e.g., there is a path from every node to every other node.

A face is a partition of the plane isolated by a set of connected vertices.

- A neighbor set is a counter-clockwise listing of the vertices incident to

a particular vertex.

- A planar embedding is a class of planar graph drawings with the same neighbor sets for each vertex. A planar graph can have an exponential number of such embeddings.
- A cutvertex is any node that causes the graph to be disconnected if it is removed.
- A biconnected graph is one without a cutvertex.
- A block is a maximally biconnected subgraph of a graph.

• A separating pair means two vertices whose removal causes a biconnected graph to become disconnected.

• A triconnected graph is one without a separating pair. A planar tri-connected graph has a unique embedding.

A face is a partition of the plane isolated by a set of connected vertices.

- A neighbor set is a counter-clockwise listing of the vertices incident to a particular vertex.
- A planar embedding is a class of planar graph drawings with the same neighbor sets for each vertex. A planar graph can have an exponential number of such embeddings.
- A cutvertex is any node that causes the graph to be disconnected if it is removed.
- A biconnected graph is one without a cutvertex.
- A block is a maximally biconnected subgraph of a graph.

• A separating pair means two vertices whose removal causes a biconnected graph to become disconnected.

• A triconnected graph is one without a separating pair. A planar tri-connected graph has a unique embedding.

A face is a partition of the plane isolated by a set of connected vertices.

- A neighbor set is a counter-clockwise listing of the vertices incident to a particular vertex.
- A planar embedding is a class of planar graph drawings with the same neighbor sets for each vertex. A planar graph can have an exponential number of such embeddings.
- A cutvertex is any node that causes the graph to be disconnected if it is removed.
- A biconnected graph is one without a cutvertex.
- A block is a maximally biconnected subgraph of a graph.

• A separating pair means two vertices whose removal causes a biconnected graph to become disconnected.

• A triconnected graph is one without a separating pair. A planar tri-connected graph has a unique embedding.

If the graph contains more cycles after removing the edges of the original cycle, this means that one or more of the pieces contains a cycle (see the purple piece). In this case, we create a subgraph containing this piece and a section of the original cycle connecting the end points of the part, and recursively call the planarity test algorithm. The pseudocode for this algorithm is as follows. Note that a separating cycle is one that generates at

least two pieces.

Given a biconnected graph G and a separating cycle C :

Compute all the pieces of G with respect to C .

1. For each piece P that is not a simple path (e.g., that contains a cycle).

(a) Create graph G consisting of P plus C .

(b) Create cycle C consisting of a path through P plus the section of C joining the ends.

(c) Apply the algorithm to (G, C) .

If the result is nonplanar

Once a graph has been either determined to be planar or has been augmented to achieve planarity, there are many possible strategies for generating

a drawing. One such technique, called the visibility approach consists of a two-step process. In the first step, called the visibility step, a visibility representation of the graph is formed. In such a representation, each vertex

is depicted as a horizontal line segment, and each edge is depicted as a vertical line connecting the corresponding vertex segments. It should be clear

that for a planar graph, it is always possible to draw such a representation without crossing edges other than where they meet the vertex segments. Obviously, many possible orderings of the vertex segments are possible; one strategy would be to arrange them to minimize the total length of the vertical connectors.

In the second step, called the replacement step, each vertex segment is collapsed to a single point, and each vertical connector is replaced by a polyline that follows the original edge as much as possible, with a segment at each end connecting the edge to its corresponding vertex. Many options exist for the replacement step, including the location of the nodes and the strategy used to form the connections (e.g., straight versus curved lines, single segment versus multiple segments).

describes an analysis of multiplex networks. He defines a measure of the intertwining of edges (entanglement index) which leads to other measures (e.g., intensity and homogeneity) and finally leads to catalyst interaction network understanding.

Networks are quite varied. Complex networks have been studied for a long time, and their characterization and definition is still the subject of We can, however, admit that a complex network is

Visualization Techniques for Trees, Graphs, and Networks

F The stages of drawing a planar graph. From left to right: original graph, visibility

representation, and replacement step.

a network which presents characteristics we cannot find in simple networks nor in random networks, but that we very often find in real-world networks More empirically, the literature has uncovered and still uncovers

complex network features in many “if not all” real-world networks.

Of such characteristics the most famous are a small world appearance—the diameter of the network is rather limited in comparison with one of a random graph with the same number of nodes and edges; a scale-free degree distribution—some observe a long tail in distribution of nodes degree, often referenced as power law; or a high clustering coefficient—the coefficient is

much higher for real-world networks than for random networks. A complex network presents at least some of these features—which are not limited to the ones depicted previously. In a real-world graph, we can also observe many disconnected components with a lot of variation in their size (probably induced by some inner scale-free feature as described above). One or more largest components may present most of the features of complex networks. Strogatz in his work defines six different ways for networks to display complexity:

- structural complexity (edges are tangled),
- network evolution (the network evolves over time),
- connection diversity (weights/directions/signs of edges),
- dynamical complexity (node states can vary with time),
- node diversity (different types of nodes),
- meta-complication.

9.2. Displaying Arbitrary Graphs/Networks

a • • •

b • • •

c • • • •

d • • •

e • • •

f • • •

g • • •

h • • •

p q r s t u v w

p • • •

q • • •

r • • •

s • • • •

t • • • •

u • • •

v • • •

w • • •

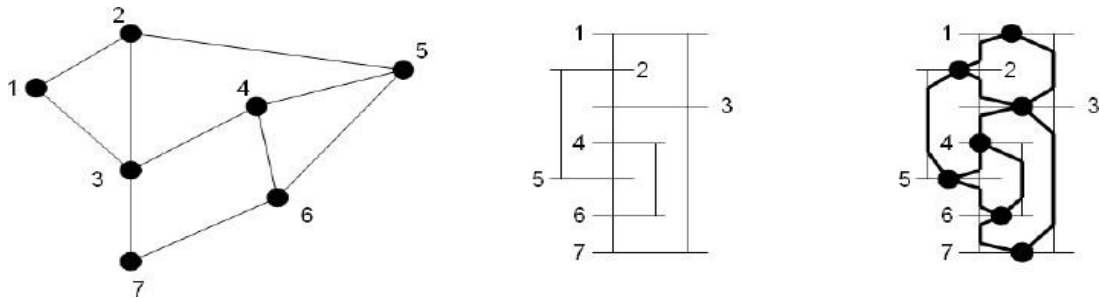
Two matrix displays of the same graph, using different orderings of nodes. Structure is more clearly present in the matrix on the right.

Matrix Representations for Graphs

An alternate visual representation of a graph is via an adjacency matrix, An alternate visual representation of a graph is via an adjacency matrix,

which is an N by N grid (where N is the number of nodes), where position (i, j) represents the existence (or not) of a link between nodes i and j .

This may be a binary matrix, or the value might represent the strength or weight of the link between the two nodes. This method overcomes one of the biggest problems with node-link diagrams, namely that of crossing edges, though it doesn't scale well to graphs with large numbers (thousands) of nodes. Bertin was one of the first researchers to investigate the power of this representation, using different reordering strategies to organize the rows and columns to reveal structures within the graph. The importance of the reordering is apparent in Figure where each matrix represents the same eight-node graph. The two four-node cliques are clearly apparent in



4 . Text and Document Visualization:

We now have huge resources of information; from libraries, to e-mail archives, to all facets of applications running on the World Wide Web. Visualization is a great aid in analyzing such data. We can visualize in many different ways things such as a blog, a wiki, a twitter feed, billions of words, a collection of papers, or a digital library. Since visualizations are task dependent, we can look at what tasks are necessary for dealing with text, documents, or web-based objects. For text and documents, the most obvious tasks are searching for a word, phrase, or topic. For partially structured data, we may search for relationships between words, phrases, topics, or documents. For structured text or document collections, the key task is most often searching for patterns and outliers within the text or documents.

4.1 Introduction:

We define a collection of documents as a corpus (plural corpora). We deal with objects within corpora. These objects can be words, sentences, paragraphs, documents, or even collections of documents. We may even consider images and videos. Often these objects are considered atomic with respect to the task, analysis, and visualization. Text and documents are often minimally structured and may be rich with attributes and metadata, especially when focused in a specific application domain. For example, documents have a format and often include metadata about the document (i.e., author, date of creation, date of modification, comments, size). Information retrieval systems are used to query corpora, which requires computing the relevance of a document with respect to a query. This requires document preprocessing and interpretation of the semantics of text. We can compute statistics about documents.

We can compute statistics about documents. For example, the number of words or paragraphs, or the word distribution or frequency, all can be used for author authenticity. Are there any paragraphs that repeat the same words or sentences? We can also identify relationships between paragraphs or documents within a corpus. For example, one could ask, "What documents relate to the spread of flu?" This is not a simple query; it isn't simply searching for the word flu. Why? We can further look for natural connections or relationships between various documents. What clusters are there? Do they represent themes within the corpus? Similarity could be defined in terms of citations, common authorships, topics, and so on.

4.2 Levels of Text Representations:

We define three levels of text representation: lexical, syntactic, and semantic. Each requires us to convert the unstructured text to some form of structured data.

Lexical level. The lexical level is concerned with transforming a string of characters into a sequence of atomic entities, called tokens. Lexical analyzers process the sequence of characters with a given set of rules into a new sequence of tokens that can be used for further analysis. Tokens can include characters, character n-grams, words, word stems, lexemes, phrases, or word n-grams, all with associated attributes. Many types of rules can be used to extract tokens, the most common of which are finite state machines defined by regular expressions.

Syntactic level. The syntactical level deals with identifying and tagging (an notating) each token's function. We assign various tags, such as sentence position or whether a word is a noun, expletive, adjective, dangling modifier, or conjunction. Tokens can also have attributes such as whether they are singular or plural, or their proximity to other tokens. Richer tags include date, money, place, person, organization, and time (Figure 10.3). The process of extracting these annotations is called named entity recognition (NER). The richness and wide variety of language models and grammars (generative, categorical, dependency, probabilistic, and functionalist) yield a wide variety of approaches.

Semantic level. The semantic level encompasses the extraction of meaning and relationships between pieces of knowledge derived from the structures identified in the syntactical level. The goal of this level is to define an analytic interpretation of the full text within a specific context, or even independent of context.

4.3 The Vector Space Model:

Computing term vectors is an essential step for many document and corpus visualization and analysis techniques. In the vector space model [356], a term vector for an object of interest (paragraph, document, or document collection) is a vector in which each dimension represents the weight of a given word in that document. Typically, to clean up noise, stop words (such as "the" or "a") are removed (filtering), and words that share a word stem are aggregated together (.

The pseudocode below counts occurrences of unique tokens, excluding stop words. The input is assumed to be a stream of tokens generated by a lexical analyzer for a single document. The terms variable contains a hashtable that maps unique terms to their counts in the document.

```
Count-Terms(tokenStream)
1 terms ← ∅ initialize terms to an empty hashtable.
2 for each token t in tokenStream
3     do if t is not a stop word
4         do increment (or initialize to 1) terms[t]
5 return terms
```

We can apply the pseudocode to the following text.

There is a great deal of controversy about the safety of genetically engineered foods. Advocates of biotechnology often say that the risks are overblown. "There have been 25,000 trials of genetically modified crops in the world, now, and not a single incident, or anything dangerous in these releases," said a spokesman for Adventa Holdings, a UK biotech firm. During the 2000 presidential campaign, then-candidate George W. Bush said that "study after study has shown no evidence of danger." And Clinton Administration Agriculture Secretary Dan Glickman said that "test after rigorous scientific test" had proven the safety of genetically engineered products.

The paragraph contains 98 string tokens, 74 terms, and 48 terms when stop words are removed. Here is a sample of the term vector that would be generated by the pseudocode:

genetically	said	safety	engineered	study	test	great	deal	controversy	foods
3	3	2	2	2	2	1	1	1	1

4.3.1 Computing Weights:

This vector space model requires a weighting scheme for assigning weights to terms in a document. There exist many such methods, the most well known of which is the term frequency inverse document frequency (tf-idf).

Let $Tf(w)$ be the term frequency or number of times that word w occurred in the document, and let $Df(w)$ be the document frequency (number of documents that contain the word).

Let N be the number of documents.

We define $TfIdf(w)$ as:

$$TfIdf(w) = Tf(w) * \log \left(\frac{N}{Df(w)} \right).$$

This is the relative importance of the word in the document, which matches our intuitive view of the importance of words. A word is more important the fewer documents it appears in (lower Df), as well as if it appears several times in a single target document (larger Tf). Said another way, we are more interested in words that appear often in a document, but not often in the collection. Such words are intuitively more important, as they are differentiating, separating or classifying words.

4.3.2 Zipf's Law:

The normal and uniform distributions are the ones we are most familiar with. The power law distribution is common today with the large data sizes we encounter, which reflect scalable phenomena. The economist Vilfredo Pareto stated that a company's revenue is inversely proportional to its rank—a classic power law, resulting in the famous 80-20 rule, in which 20% of the population holds 80% of the wealth.

Harvard linguist George Kingsley Zipf stated the distribution of words in natural language corpora using a discrete power law distribution called a Zipfian distribution. Zipf's Law states that in a typical natural language document, the frequency of any word is inversely proportional to its rank in the frequency table. Plotting the Zipf curve Fig 10.2 on a log-log scale yields a straight line with a slope of -1.

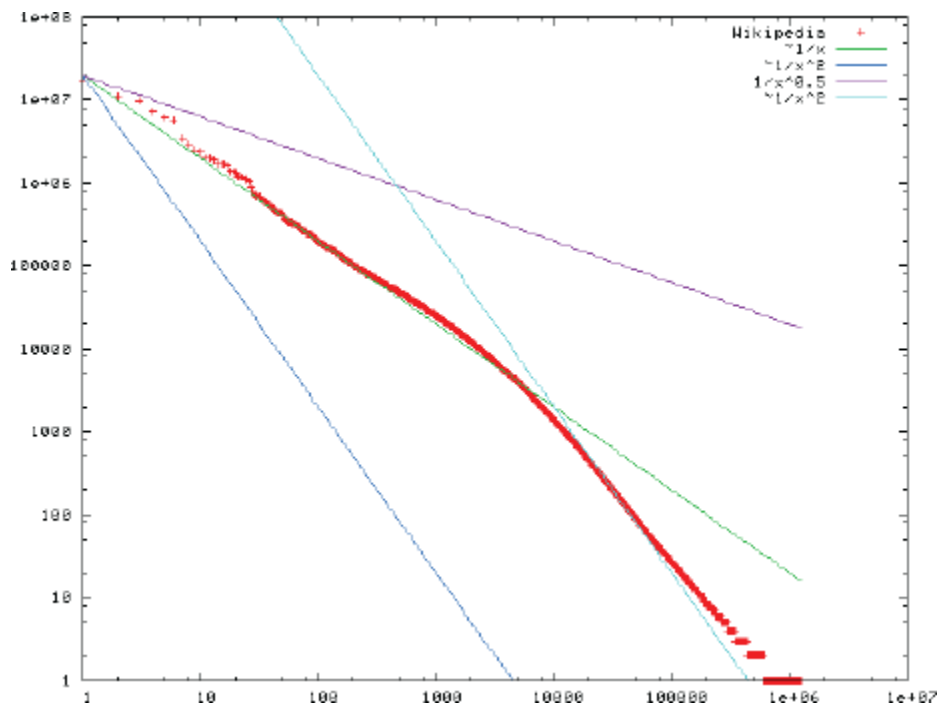


Figure 10.2. The distribution of terms in Wikipedia, an example of Zipf's Law in action. Term frequency is on the y-axis, and frequency rank is on the x-axis.

One immediate implication of Zipf's Law is that a small number of words describe most of the key concepts in small documents. There are numerous examples of text summarization that permit a full description with just a few words.

4.3.3 Tasks Using the Vector Space Model :

The vector space model, when accompanied by some distance metric, allows one to perform many useful tasks. We can use tf-idf and the vector space model to identify documents of particular interest. For example, the vector space model, with the use of some distance metric, will allow us to answer questions such as which documents are similar to a specific one, which documents are relevant to a given collection of documents, or which documents are most relevant to a given search query—all by finding the documents whose term vectors are most similar to the given document, the average vector over a document collection, or the vector of a search query.

Another indirect task is how to help the user make sense of an entire corpus. The user may be looking for patterns or for structures, such as a document's main themes, clusters, and the distribution of themes through a document collection. This often involves visualizing the corpus in a two dimensional layout, or presenting the user with a graph of connections between documents or entities to navigate through. The visualization pipeline maps well to document visualization: we get the data (corpus), transform it into vectors, then run algorithms based on the tasks of interest (i.e., similarity, search, clustering), and generate the visualizations.

4.3.4 Single Document Visualizations:

Here we present several visualizations of a single text document, taken from the VAST Contest 2007 data set.

4.4.5 Literature Fingerprinting:

Literature fingerprinting is a method of visualizing features used to characterize text. Instead of calculating just one feature value or vector for the whole text (this is what is usually done), we calculate a sequence of feature values per text and present them to the user as a characteristic fingerprint of the document. This allows the user to “look inside” the document and analyze the development of the values across the text. Moreover, the structural information of the document is used to visualize the document on different levels of resolution. Literature fingerprinting was applied to an authorship attribution problem to show the discrimination power of the standard measures that are assumed to capture the writing style of an author .

4.5 Document Collection Visualization:

In most cases of document collection visualizations, the goal is to place similar documents close to each other and dissimilar ones far apart. This is a minimax problem and typically $O(n^2)$. We compute the similarity between all pairs of documents and determine a layout. The common approaches are graph spring layouts, multidimensional scaling, clustering (k-means, hierarchical, expectation maximization (EM), support vector), and self-organizing maps. We present several document collection visualizations, such as self organizing maps, cluster maps, and themespaces.

4.5.1 Self-Organizing Maps:

A self-organizing map (SOM) is an unsupervised learning algorithm using a collection of typically 2D nodes, where documents will be located. Each node has an associated vector of the same dimensionality as the input vectors (the document vectors) used to train the map. We initialize the SOM nodes, typically with random weights. We choose a random vector from the input vectors and calculate its distance from each node.

We adjust the weights of the closest nodes (within a particular radius), making each closer to the input vector, with the higher weights corresponding to the closest selected node. As we iterate through the input vectors, the radius gets smaller. An example of using SOMs for text data is shown in Figure , which shows a million documents collected from 83 newsgroups.

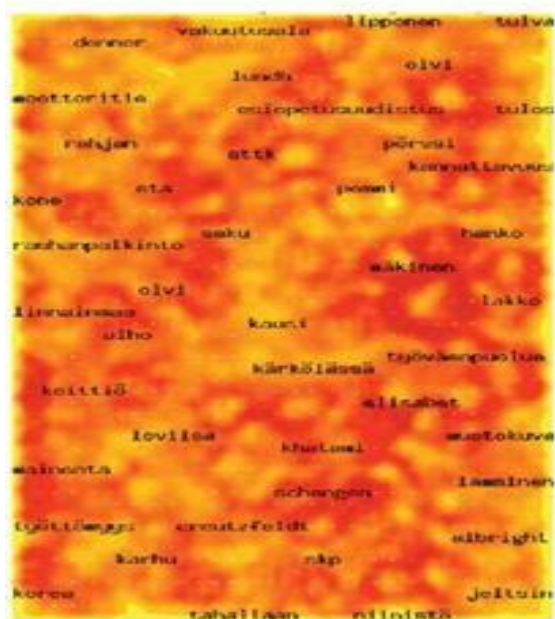


Fig: A self-organizing map (SOM) layout of Finnish news bulletins. The labels show the topical areas, and color represents the number of documents, with light areas containing more

4.5.2 Themescapes:

Themescapes are summaries of corpora using abstract 3D landscapes in which height and color are used to represent density of similar documents. The example shown in Figure from Pacific Northwest National Labs [407] represents news articles visualized as a themescape. The taller mountains represent frequent themes in the document corpus (height is proportional to number of documents relating to the theme)



Fig: A themescape from PNNL that uses height to represent the frequency of themes in news articles.

4.5.3 Document Cards:

Document cards are a compact visualization that represents the document's key semantics as a mixture of images and important key terms, similar to cards in a top trumps game. The key terms are extracted using an advanced text-mining approach based on an automatic extraction of document structure. The images and their captions are extracted using a graphical heuristic, and the captions are used for a semi semantic image weighting. Furthermore, the image color histogram is used to classify images into classes (class 1: photography/rendered image, class 2: diagram/sketch/graph, class 3: table) and show at least one representative from each non-empty class.

4.6 Extended Text Visualizations

4.6.1 Software Visualization

Eick et al. developed a visualization tool called SeeSoft that visualizes statistics for each line of code (i.e., age and number of modifications, programmer, dates). In Figure , each column represents a source code file with the height representing the size of the file. If the file is longer than the screen, it continues into the next column. In the classic SeeSoft representation, each row represents one line of code. Since the number of lines is too large for one row, each line of code is represented by a pixel in the row. This increases the number of lines that can be displayed. Color is used to represent the call count. The more red a line is, the more often the line is called, and thus is a key hot-spot. A blue line

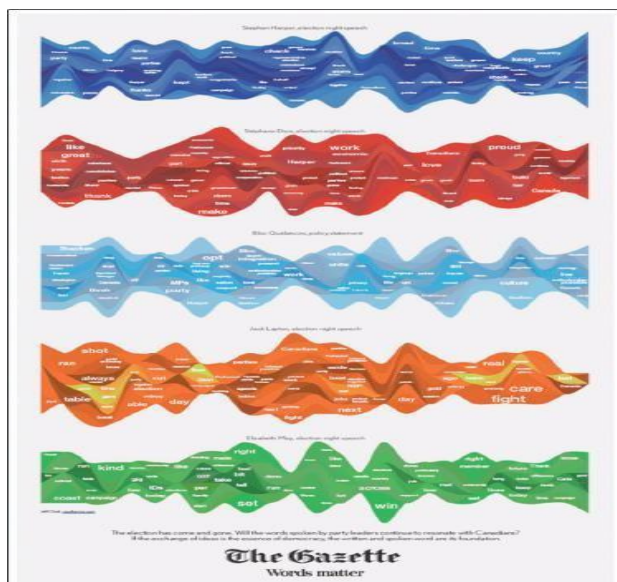
is an infrequently called one. Color can be used to represent other parameters, such as time of last modification or number of modifications. With a $1K \times 1K$ screen, SeeSoft is able to display up to 50,000 lines of code. This figure contains 52 files with 15,255 lines of code. The selected file is file1.c, a block of code with a zoomed-in view of line 408.

4.6.2 Search Result Visualization

Marti Hearst developed a simple query result visualization foundationally similar to Keim's pixel displays called TileBars, which displays a number of term-related statistics, including frequency and distribution of terms, length of document, term-based ranking, and strength of ranking. Each document of the result set is represented by a rectangle, where width indicates relative length of the document and stacked squares correspond to text segments. Each row of the stack represents a set of query terms, and the darkness of the square indicates the frequency of terms among the corresponding terms. Titles and the first words from the document appear next to its TileBar. Each large rectangle indicates a document, and each square within the document represents a text segment. The darker the tile, the more frequent is the query term set. This produces a representation that is compact and provides feedback on document structure reflecting relative document length, query term frequency, and query term distribution.

4.6.3 Temporal Document Collection Visualizations

ThemeRiver, also called a stream graph, is a visualization of thematic changes in a document collection over time. This visualization assumes that the input data progresses over time. Themes are visually represented as colored horizontal bands whose vertical thickness at a given horizontal location represents their frequency at a particular point in time.



A stream graph (ThemeRiver), depicting the election night speeches of several different candidates for a Canadian election.

Jigsaw is a tool for visualizing and exploring text corpora. Jigsaw's calendar view positions document objects on a calendar based on date entities identified within the text. When the user highlights a document, the entities that occur within that document are displayed.

Wanner et al. developed a visual analytics tool for conducting semiautomatic sentiment analysis of large news feeds . While the tool automatically retrieves and analyzes RSS feeds with respect to positive and negative opinion words, the more demanding news analysis of finding trends, spotting peculiarities, and putting events into context is left to the human expert. each single news item is represented by one visual object and plotted on a horizontal time axis according to its publication time. The shape and color of an item reveal information about the category it belongs to, and its vertical shift indicates whether it has a positive connotation (upward shift) or a negative one (downward shift).

4.6.4 Representing Relationships

Jigsaw also includes an entity graph view , in which the user can navigate a graph of related entities and documents. In Jigsaw, entities are connected to the documents in which they appear. The Jigsaw graph view does not show the entire document collection, but it allows the user to incrementally expand the graph by selecting documents and entities of interest.

